

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 11. Tuple

Lecture: 124. Packing and Unpacking

Section 1: Lecture Summary

This lecture covers several operations that can be performed on Python tuples, focusing on concatenation, repetition, packing, unpacking, and membership operators. The instructor demonstrates how tuples behave similarly to lists and strings with concatenation (using `+`) and repetition (using `*`), both producing new tuples without modifying the originals. The central topic is packing and unpacking: packing is when multiple values are assigned to a single variable, creating a tuple; unpacking is the reverse—assigning tuple elements to multiple variables. The lecture highlights the requirement that the count of variables must match the tuple length during unpacking unless the starred expression `*` is used. Starred unpacking allows one variable to capture multiple (rest) elements as a list. The membership operators `'in'` and `'not in'` also work with tuples and allow checking if an element exists in a tuple. Examples are provided in Jupyter Notebook format, demonstrating all concepts interactively.

Section 2: Key Concepts and Explanations

- Tuple Concatenation
 - Operator: `+`
 - Combines two tuples by appending all elements of the second to the first.
 - Produces a new tuple; original tuples remain unchanged.
- Tuple Repetition
 - Operator: `*`
 - Repeats the entire tuple a specified number of times.
 - Produces a new tuple with repeated elements.

- Packing
 - Assigning multiple comma-separated values to a single variable.
 - Python automatically packs these values into a tuple.
 - Works regardless of data type homogeneity (can be mixed types).
- Unpacking
 - Assigning a tuple (or any sequence) to multiple variables.
 - Each variable receives one element of the tuple.
 - The number of variables must match the number of tuple elements.
 - Error if counts mismatch: `ValueError: too many values to unpack`.
- Starred Unpacking (`\*)`
 - Allows unpacking with an operator `\*` before a variable.
 - Captures "the rest" of the values (remaining elements) as a list.
 - Can be used to unpack variable-length tuples without errors.
 - Works when starred variable is at the start, middle, or end of the target variables list.
- Membership Operators
 - `in`: Checks if a value is in a tuple, returns True/False.
 - `not in`: Checks if a value is not in a tuple, returns True/False.
 - These work with all sequence data types, including tuples.

Section 3: Example Code and Use Cases

Tuple concatenation

```
t1 = (1, 2, 3)
t2 = (8, 9, 10)
t3 = t1 + t2
print(t3) # Output: (1, 2, 3, 8, 9, 10)
```

Concatenates t1 and t2 into a new tuple t3.

Tuple repetition

```
t1 = (1, 2, 3)
t2 = t1 * 3
print(t2) # Output: (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

Repeats tuple t1 three times to create t2.

Packing: multiple values assigned to single variable form a tuple

```
t1 = 1, 2, 3, 4, 5
print(t1) # Output: (1, 2, 3, 4, 5)
print(type(t1)) # Output: <class 'tuple'>
```

Assigning multiple comma-separated values automatically packs them as a tuple.

Unpacking: tuple assigned to multiple variables

```
a, b, c, d, e = t1
print(a, b, c, d, e) # Output: 1 2 3 4 5
```

Each element from the tuple t1 is unpacked into a dedicated variable.

Unpacking error example: mismatch number of variables

```
try:
    a, b, c = t1 # t1 has 5 values, but variables are 3
except ValueError as ve:
    print(ve) # Output: too many values to unpack (expected 3)
```

Number of variables must equal the number of tuple elements for unpacking.

Starred unpacking example (rest values captured as list)

```
a, b, *c = t1
print(a) # Output: 1
print(b) # Output: 2
print(c) # Output: [3, 4, 5]
```

`c` captures all elements after `a` and `b` as a list.

Starred unpacking: star at start (captures rest values)

```
*a, b, c = t1
print(a) # Output: [1, 2, 3]
print(b) # Output: 4
print(c) # Output: 5
```

`a` captures multiple values at the start.

Starred unpacking: star in middle

```
a, *b, c = t1
print(a) # Output: 1
print(b) # Output: [2, 3, 4]
print(c) # Output: 5
```

`b` captures middle elements.

Membership operator example

```
t1 = (1, 2, 3, 4, 5)
print(3 in t1) # Output: True
print(10 not in t1) # Output: True
```

Check if elements exist or do not exist in a tuple.

Section 4: Key Takeaways

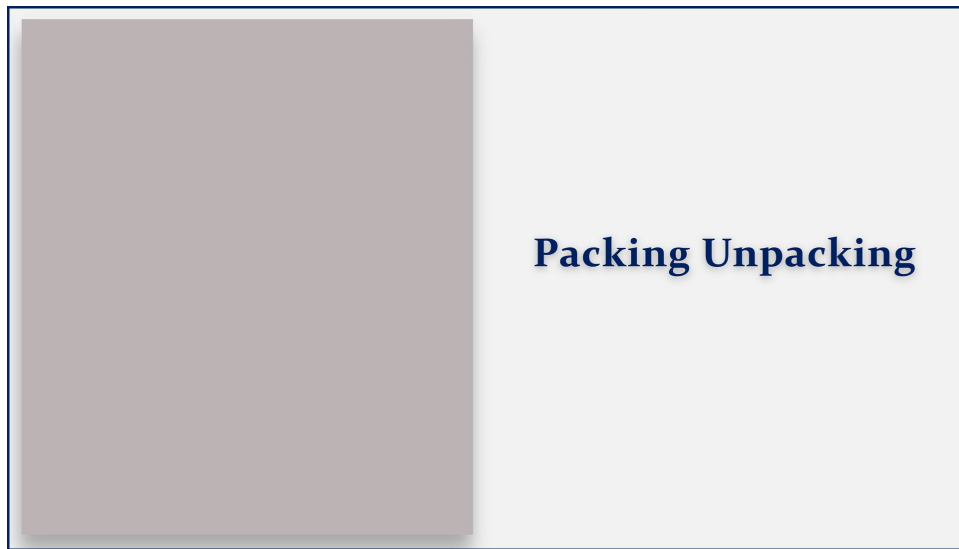
- Tuples support concatenation (+) and repetition () that create new tuples without modifying originals.
- Packing lets Python package multiple values into a tuple when assigned to a single variable.
- Unpacking extracts tuple elements into multiple variables; requires matching elements and variables count.
- Starred unpacking () allows handling variable number of elements by collecting extras as a list.
- Membership operators `in` and `not in` cleanly check presence or absence of values within tuples.

- Packing and unpacking also apply to strings, lists, and other sequence types, making these versatile operations.

- Understanding packing, unpacking, and starred unpacking simplifies handling data stored in tuples.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Packing Unpacking

Concatenation +

Packing Unpacking

Concatenation +

Repetition

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

Packing Unpacking

Concatenation

+

Repetition

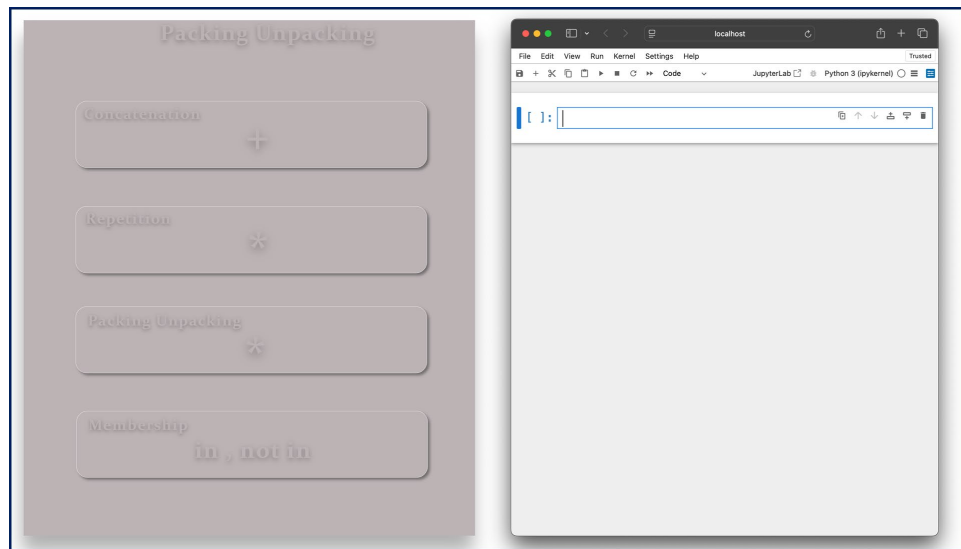
*

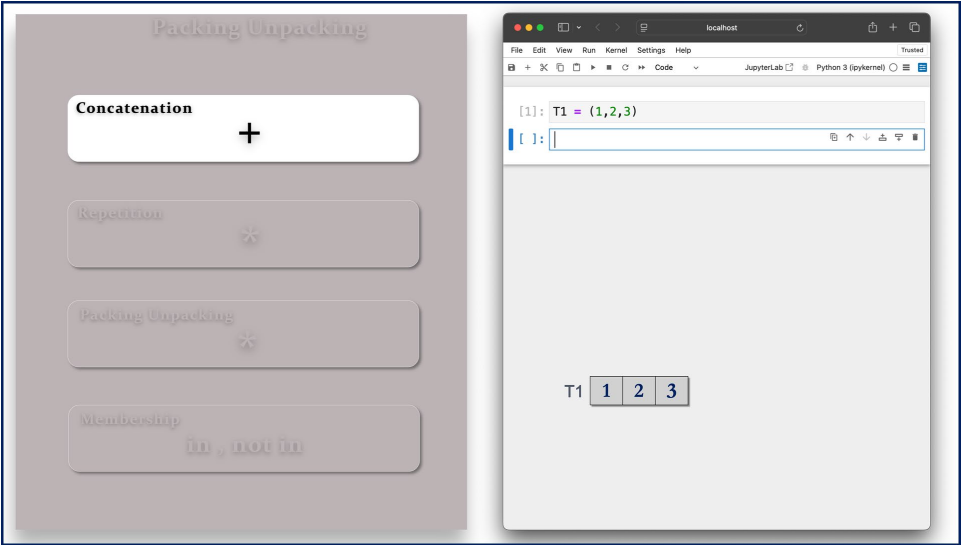
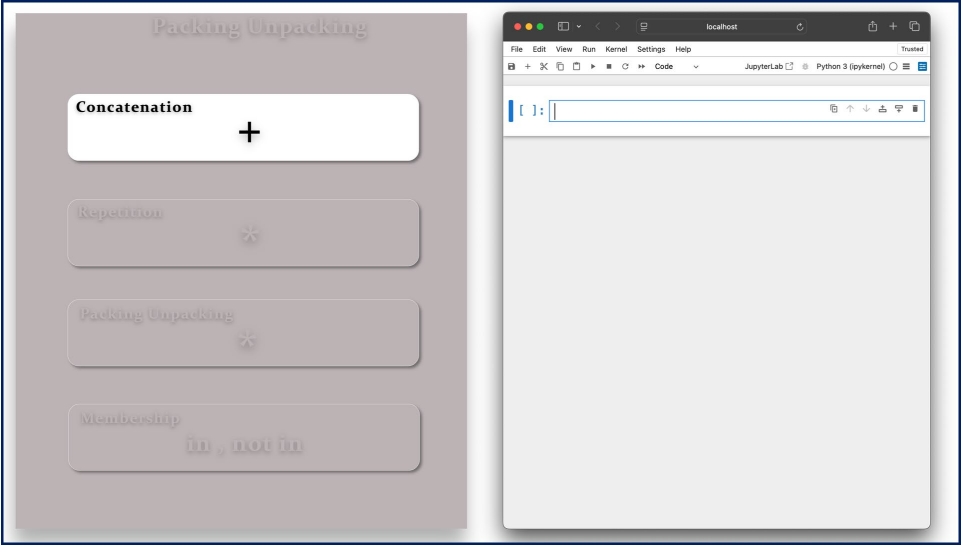
Packing Unpacking

*

Membership

in , not in





Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[2]: T2 = (8,9,10)

[]:

T1 1 2 3 T2 8 9 10

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[2]: T2 = (8,9,10)

[3]: T3 = T1 + T2

[]:

T1 1 2 3 + T2 8 9 10

T3 1 2 3 8 9 10

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

```
[1]: T1 = (1,2,3)
[2]: T2 = (8,9,10)
[3]: T3 = T1 + T2
[4]: T3
[4]: (1, 2, 3, 8, 9, 10)
[ ]:
```

T1

1

2

3

+

T2

8

9

10

T3

1

2

3

8

9

10

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

```
[ ]:
```

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[]:

T1 1 2 3

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[]: T2 = T1 * 3

T1 1 2 3

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[2]: T2 = T1 * 3

[]:

T1

1 2 3

T2

1 2 3 1 2 3 1 2 3

Packing Unpacking

Concatenation

+

Repetition

*

Packing Unpacking

*

Membership

in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = (1,2,3)

[2]: T2 = T1 * 3

[3]: T2

[3]: (1, 2, 3, 1, 2, 3, 1, 2, 3)

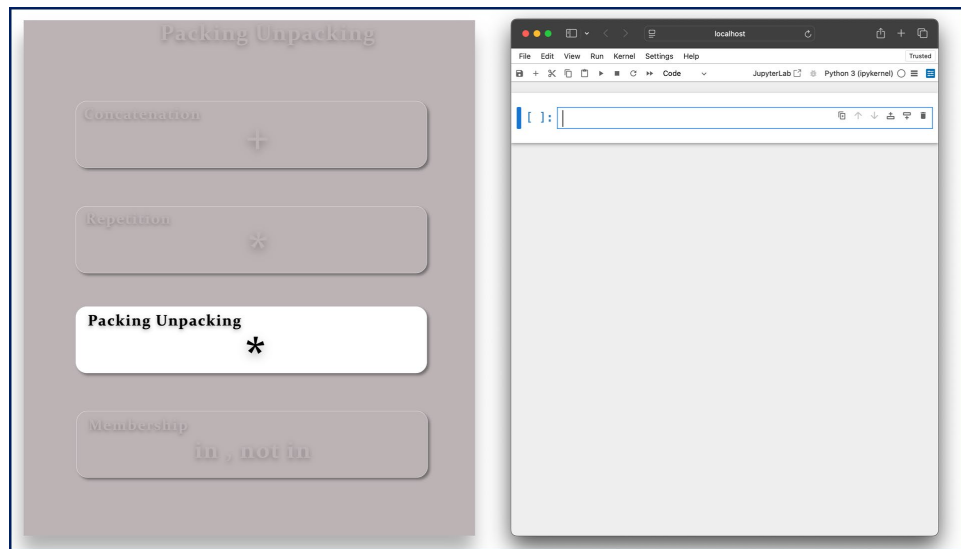
[]:

T1

1 2 3

T2

1 2 3 1 2 3 1 2 3



Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[]: T1 = 1,2,3,4,5

T1 = 1, 2, 3, 4, 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[]:

T1

1	2	3	4	5
---	---	---	---	---

T1 = 1, 2, 3, 4, 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[2]: T1

[2]: (1, 2, 3, 4, 5)

[]:

T1 1 2 3 4 5

T1 = 1, 2, 3, 4, 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[]:

T1 1 2 3 4 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[]:

T1 1 2 3 4 5

a, b, c, d, e =

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[]:

T1 1 2 3 4 5

a, b, c, d, e = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
```

```
[ ]:
```

T1

1	2	3	4	5
---	---	---	---	---

a, b, c, d, e = 1, 2, 3, 4, 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
```

```
[3]: a,b,c,d,e = T1
```

```
[ ]:
```

T1

1	2	3	4	5
---	---	---	---	---

a, b, c, d, e = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[3]: a,b,c,d,e = T1

[4]: print(a,b,c,d,e)

1 2 3 4 5

[]:

T1 1 2 3 4 5

a, b, c, d, e = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[]:

T1 1 2 3 4 5

a, b, c =

The image is a composite of two parts. The left part is a conceptual diagram titled 'Packing Unpacking' with a light purple background. It contains four rounded rectangular boxes, each with a title and a symbol: 'Concatenation' with a plus sign (+), 'Repetition' with an asterisk (*), 'Packing Unpacking' with an asterisk (*), and 'Membership' with 'in' and 'not in' symbols. The right part is a screenshot of a JupyterLab interface. The top bar shows 'localhost' and 'Trusted'. The menu bar includes 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. The toolbar has icons for file operations and a 'Code' button. The code editor shows the line `[1]: T1 = 1,2,3,4,5` with a blue cursor at the end. Below the code editor, the output is displayed as a horizontal list: `1 2 3 4 5`. Below the output, the variable assignment `a, b, c = T1` is shown in a separate box.

Packing Unpacking *

The screenshot shows a JupyterLab window with a code cell. The code cell contains the following code:

```
[1]: T1 = 1,2,3,4,5
```

The output of the code cell is a list: `[1]`. The list contains the elements `1`, `2`, `3`, `4`, and `5`.

The screenshot shows the JupyterLab application running in a web browser. The top bar includes standard window controls and a 'localhost' address bar. The main interface is divided into two panes. The left pane contains a file browser showing a directory structure with 'code' and 'data' folders. The right pane is the active workspace, which includes a code editor and a terminal. The code editor displays the command `T1 = 1,2,3,4,5` on a single line. The terminal below it shows a shell prompt `jupyterlab:~$` and a cursor waiting for input.

T1

1	2	3	4	5
---	---	---	---	---

a, b, c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
[2]: a,b,c = T1
```

Traceback
ValueError
(most recent call last)
Cell In[2], line 1
----> 1 a,b,c = T1
ValueError: too many values to unpack (expected 3)

T1 1 2 3 4 5

a, b, c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
[3]: a,b,*c = T1
```

T1 1 2 3 4 5

a 1 b 2 c 3 4 5

a, b, *c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
[3]: a,b,*c = T1
[4]: print(a,b,c)
1 2 [3, 4, 5]
```

T1 [1][2][3][4][5]

a [1] b [2] c [3][4][5]

a, b, *c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

```
[1]: T1 = 1,2,3,4,5
[2]: *a,b,c = T1
```

T1 [1][2][3][4][5]

a [1][2][3] b [4] c [5]

*a, b, c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[2]: *a,b,c = T1

[3]: print(a,b,c)

[1, 2, 3] 4 5

[]:

T1 1 2 3 4 5

a 1 2 3 b 4 c 5

*a, b, c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[4]: a,*b,c = T1

[]:

T1 1 2 3 4 5

a 1 b 2 3 4 c 5

a, *b, c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: T1 = 1,2,3,4,5

[4]: a,*b,c = T1

[5]: print(a,b,c)

1 [2, 3, 4] 5

[]:

T1 1 2 3 4 5

a 1 b 2 3 4 c 5

a,*b,c = T1

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[]:

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: L1=[1,2,3,4,5]

[2]: *a,b,c=L1

[3]: print(a)
print(b)
print(c)

[1, 2, 3]
4
5

[]:

T1 1 2 3 4 5

a 1 2 3 b 4 c 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[]:

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

+

+

+

+

+

+

Code

JupyterLab

Python 3 (ipykernel)

[1]: L1=[1,2,3,4,5]

[]:

T1

1	2	3	4	5
---	---	---	---	---

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[1]: L1=[1,2,3,4,5]

[2]: a,*b,c=L1

[]:

T1 1 2 3 4 5

a 1 b 2 3 4 c 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[1]: L1=[1,2,3,4,5]

[2]: a,*b,c=L1

[3]: print(a)

print(b)

print(c)

1

[2, 3, 4]

5

[]:

T1 1 2 3 4 5

a 1 b 2 3 4 c 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: a,*b,c=range(6)

[]:

a 0 b 1 2 3 4 c 5

Packing Unpacking

Concatenation +

Repetition *

Packing Unpacking *

Membership in , not in

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: a,*b,c=range(6)

[2]: print(a)

print(b)

print(c)

0

[1, 2, 3, 4]

5

[]:

a 0 b 1 2 3 4 c 5